

TECHNICAL DESIGN DOCUMENT

Gestionale Rendicontazione

Sistema di Rendicontazione Ore

v1.3.0 — Aggiornamento allo stato reale del backend + Roadmap Frontend Blazor

16 Luglio 2026

Documento tecnico di riferimento dell'intero progetto: architettura, modello dati, protocolli e configurazione.

Indice

Indice	2
1. Intestazione Tecnica	3
2. Architettura del Progetto	4
2.1 Struttura della Solution (.sln)	4
2.2 Modello Dati — Diagramma Entità-Relazione	4
2.3 Pattern e Principi Architettureali	7
3. Protocolli D'Utilizzo	8
3.1 Convenzioni Generali	8
3.2 Endpoint	9
Auth	9
Worklog	9
Report e Log	9
Anagrafiche (Company, Employee, Project, Status, Type)	10
4. Visualizzazione Struttura Macro e Micro	11
4.1 Struttura Macro	11
4.2 Struttura Micro	11
4.2.1 Componenti del Layer API	11
4.2.2 Componenti del Layer DataAccess	11
4.2.3 Componenti del Layer Domain	11
4.2.4 Componenti del Frontend (Blazor WebAssembly)	12
4.2.5 Flusso di Generazione del Report PDF	12
5. Note Tecniche di Configurazione	13
5.1 Configurazione JWT (appsettings.json)	13
5.2 CORS (appsettings.json)	13
5.3 Connessione al database	13
5.4 Avvio e seed dati	13

1. Intestazione Tecnica

Campo	Specifica Tecnica
Nome Progetto	Gestionale Rendicontazione
Autori	Alessandro Gabriele Gregorio
Obiettivo Applicativo	Rendicontazione giornaliera e mensile delle attività lavorative dei tecnici, ai fini del controllo di gestione e della fatturazione, con generazione di report in formato PDF per tecnico o per progetto.
Stack Backend	ASP.NET Core Web API, DevExpress XPO (ORM), SQLite (ambiente di sviluppo)
Stack Frontend	Blazor WebAssembly (standalone), Blazorise Free/Community (provider Bootstrap 5, icone FontAwesome)
Libreria Generazione PDF	PDFsharp (porting WASM), generazione interamente lato client
Protocollo di Comunicazione	REST API JSON, documentate con Swagger/OpenAPI (Swashbuckle) in ambiente di sviluppo
Autenticazione	JWT Bearer Token self-issued (HMAC-SHA256), con revoca via blacklist persistita su database
Logging	Serilog, con sink su console e su database (tabella log applicativo consultabile da interfaccia Admin)

2. Architettura del Progetto

Il sistema adotta una variante pulita di Clean Architecture / DDD Light, finalizzata a isolare il nucleo di business (Domain) dalle dipendenze esterne di persistenza (DataAccess) e di esposizione (API). Il frontend Blazor WebAssembly comunica con il backend esclusivamente tramite chiamate HTTP/REST stateless: nessuna connessione persistente (SignalR/WebSocket) è prevista. Questo mantiene il backend privo di stato applicativo legato alla UI e semplifica la scalabilità orizzontale.

2.1 Struttura della Solution (.sln)

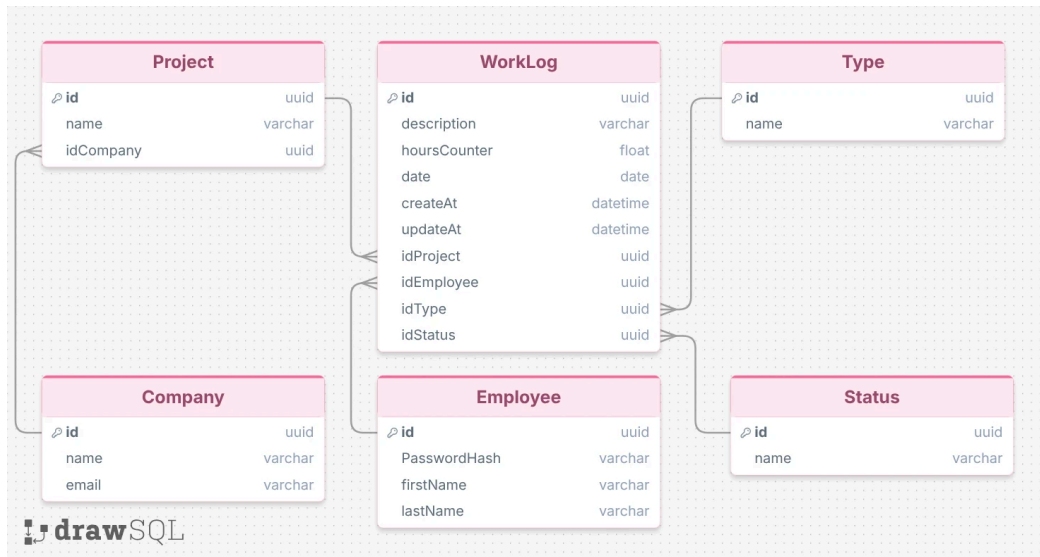
L'alberatura effettiva del codice sorgente, così come presente nel repository, è la seguente:

```
sln/
├── API/
│   ├── Controllers/
│   ├── Services/
│   ├── helpers/
│   └── Program.cs
│   # Layer di Esposizione (Presentazione Backend)
│   # Controller REST per l'esposizione degli endpoint
│   # Estensioni di bootstrap: Jwt, Xpo, Cors, Swagger
│   # ProblemDetails, enrichment log, estensioni
│   # Entry point, composizione della pipeline
├── DATAACCESS/
│   ├── Datacontext/
│   ├── Services/
│   └── helpers/
│   # Layer di Infrastruttura e Persistenza
│   # IDbContextService (Session/UnitOfWork XPO) e
│   # Implementazione dei Servizi di Business Logic
│   # WorkLogQueryExtensions (soft delete),
├── MappingProfile
├── DOMAIN/
│   ├── Entities/
│   ├── Dtos/
│   ├── Interfaces/
│   └── Constants/
│   # Core Layer
│   # Modelli del database espressi ad oggetti (XPO)
│   # Data Transfer Objects (Request/Response)
│   # Contratti per Repository e Servizi applicativi
│   # Costanti condivise (nomi dei ruoli)
├── FRONTEND/
│   └── GestionaleRendicontazione.Client
│       ├── Pages/Dashboard/
│       ├── Pages/Admin/
│       └── Layout/, Services/
│       # Area utente: worklog, report
│       # Area amministrativa: anagrafiche, log
│       # Layout condiviso, client HTTP tipizzati
```

Ogni pagina Blazor rilevante (Dashboard, Companies, Employees, Projects, Statuses, Types, Logs) è suddivisa in un file .razor per il markup e una o più classi parziali .razor.cs nella sottocartella Services/, una per ciascuna responsabilità (es. Dashboard.CreateWorklog.cs, Dashboard.EditWorklog.cs, Dashboard.DeleteWorklog.cs, Dashboard.Report.cs): questo mantiene ogni file focalizzato su una singola operazione invece di accumulare tutta la logica nel code-behind principale.

2.2 Modello Dati — Diagramma Entità-Relazione

Il modello dati riflette il dominio applicativo della rendicontazione ore. L'entità centrale WorkLog rappresenta la singola riga di consuntivazione giornaliera, collegata a Project (e di conseguenza a Company), Employee, Type e Status. La struttura delle relazioni è confermata dagli attributi [Association] di XPO nel codice.



Company

Anagrafica società cliente/interna. Relazione 1-N con Project tramite Project.Company (colonna idCompany).

Campo	Tipo / Note
id	Guid — chiave primaria XPO (autogenerata)
name	varchar, indicizzato univoco
email	varchar, indicizzato univoco

Project

Progetto associato a una Company. Relazione N-1 con Company (idCompany) e 1-N con WorkLog (idProject).

Campo	Tipo / Note
id	Guid — chiave primaria XPO
name	varchar
idCompany	FK → Company

Employee

Tecnico/dipendente che rendicontra le ore. Estende PermissionPolicyUser di DevExpress, ereditandone Oid, UserName, IsActive e la collection Roles: questa scelta fornisce il sistema di autorizzazione a ruoli (Admin/User) oggi applicato sui controller.

Campo	Tipo / Note
id	Guid — chiave primaria (ereditata da PermissionPolicyUser)
passwordHash	varchar(1024) — hash generato con Microsoft.AspNetCore.Identity.PasswordHasher
firstName	varchar
lastName	varchar
Roles	collection ereditata — determina i claim di ruolo (Admin/User) nel token JWT

Type

Tipologia di attività rendicontata (es. FIX). Relazione 1-N con WorkLog tramite WorkLog.Type (idType).

Campo	Tipo / Note
id	Guid — chiave primaria XPO
name	varchar, indicizzato univoco

Status

Stato della rendicontazione (es. WORKING_PROGRESS, REJECTED). Relazione 1-N con WorkLog tramite WorkLog.Status (idStatus).

Campo	Tipo / Note
id	Guid — chiave primaria XPO
name	varchar, indicizzato univoco

WorkLog

Entità centrale del modello: singola riga di rendicontazione ore. Relazione N-1 con Project, Employee, Type e Status.

Campo	Tipo / Note
id	Guid — chiave primaria XPO
description	varchar
hoursCounter	float
date	dateOnly
createAt / updateAt	datetime
isDeleted	bool — flag di soft delete, letto in lettura tramite WorkLogQueryExtensions.Active()
idProject / idEmployee / idType / idStatus	FK → Project / Employee / Type / Status

2.3 Pattern e Principi Architeturali

Accesso ai dati tramite DevExpress XPO. Il layer DataAccess espone un servizio unico, IDbContextService, che incapsula due modalità di accesso al database XPO, coerentemente con quanto implementato in DbContextService.cs:

- Sola lettura — `ExecuteReadOnly<T>(Func<Session, T>) / ExecuteReadOnly(Action<Session>)`: apre una Session XPO con `TrackPropertiesModifications = false` (nessun tracciamento delle modifiche, adatta a query pure), eseguita in modo sincrono.
- Lettura e scrittura — `ReadWrite(Func<UnitOfWork, Task>)`: apre una UnitOfWork XPO, esegue l'operazione passata come lambda asincrona e chiama `CommitChangesAsync()` al termine; in caso di eccezione la modifica non viene salvata.

Il IDataLayer XPO (ThreadSafeDataLayer) è registrato come Singleton in `AddXpoInfrastructure`, mentre IDbContextService è registrato come Scoped:

- Singleton — IDataLayer: creato una sola volta all'avvio dell'applicazione, con lo schema XPO generato per riflessione da `ReflectionDictionary.GetDataStoreSchema()` sull'assembly delle Entities.
- Scoped — IDbContextService: una istanza per richiesta HTTP, che internamente apre/chiude Session o UnitOfWork sul DataLayer condiviso.
- Dependency Injection — tutte le dipendenze applicative sono registrate nel container nativo di ASP.NET Core, suddivise per responsabilità in metodi di estensione dedicati (`AddXpoInfrastructure`, `AddBlazorClientCors`, `AddApiDocumentation`, `AddJwtAuthenticationWithBlacklist`, `AddApplicationServices`) richiamati da `Program.cs`.
- DTO Mapping — nessuna Entity è esposta direttamente verso l'esterno. Il mapping tra Entities e DTO è centralizzato in un profilo AutoMapper (MappingProfile) invece di essere scritto a mano in ogni service.
- Validazione — Data Annotations sui Dtos in ingresso (`[Required]`, `[StringLength]`, `[Range]`, `[EmailAddress]`), verificate tramite `ModelState.IsValid` nel Controller.
- Autorizzazione a ruoli — applicata sui controller tramite `[Authorize]` / `[Authorize(Roles = RoleNames.Admin)]`. Le anagrafiche (Company, Employee, Status, Type in scrittura) sono riservate al ruolo Admin; WorklogController distingue internamente tra vista Admin (tutti i dipendenti, filtri liberi) e vista utente (solo i propri worklog, con `IdEmployee` sempre forzato al claim del token, mai accettato dal body).
- Revoca token — i JWT self-issued sono per natura stateless, ma il logout inserisce il jti del token in una blacklist persistita su database (`BlacklistedToken`); ogni richiesta autenticata verifica l'appartenenza alla blacklist prima di considerare valido il token.
- Gestione errori centralizzata — un middleware dedicato (`UseGlobalProblemDetails`) intercetta le eccezioni non gestite e le mappa a risposte in formato ProblemDetails (RFC 7807), con status code differenziato per tipo di eccezione (`InvalidOperationException` → 409, `UnauthorizedAccessException` → 401, `ArgumentException` → 400, default → 500) e dettagli aggiuntivi (`traceId`, `stackTrace`) solo in ambiente di sviluppo.
- Logging strutturato — Serilog arricchisce ogni evento con `UserId/RequestPath/RequestMethod` (`RequestContextEnricher`) e scrive in parallelo su console e su database (`XpoSerilogSink`), consultabile dalla pagina Admin › Logs con filtri per testo, livello, metodo e intervallo di date.
- Separation of Concerns per la generazione PDF — il backend espone solo dati aggregati in JSON (endpoint `/api/report-data`); la composizione del PDF vero e proprio avviene interamente nel browser, lato Blazor WebAssembly.

3. Protocolli D'Utilizzo

La comunicazione tra Frontend e Backend avviene tramite API RESTful, con payload in formato JSON. Il backend è raggiungibile ed esplorabile anche tramite Swagger UI (Swashbuckle), attivo in ambiente di sviluppo.

3.1 Convenzioni Generali

- Versionamento — tutti gli endpoint sono esposti sotto il prefisso /api/.
- Formato dati — tutte le risposte del backend sono in formato application/json.
- Autenticazione — JWT Bearer Token self-issued (HMAC-SHA256), inserito nell'header Authorization su ogni richiesta autenticata. Il token è firmato con una chiave simmetrica letta da appsettings.json (sezione Jwt), scadenza configurabile (Jwt:ExpiryMinutes, default 60 minuti).
- CORS — le origini consentite (tipicamente gli indirizzi di sviluppo del client Blazor) sono elencate in appsettings.json (sezione Cors:AllowedOrigins) e applicate come primo middleware della pipeline, prima di qualunque middleware in grado di generare direttamente una risposta (Swagger compreso).
- Codici di stato HTTP — 200 OK, 201 Created, 204 No Content, 400 Bad Request (validazione), 401 Unauthorized (token assente/non valido/in blacklist), 403 Forbidden (ruolo insufficiente), 404 Not Found, 409 Conflict (regola di business violata), 422 Unprocessable Entity (FK inesistenti o vincoli applicativi).
- Gestione errori — risposta in formato ProblemDetails (RFC 7807) sia per le validazioni (ValidationProblem) sia per le eccezioni non gestite, tramite il middleware globale descritto in §2.3.
- Documentazione interattiva — Swagger / OpenAPI (Swashbuckle.AspNetCore) abilitato in ambiente di sviluppo.

3.2 Endpoint

Auth

Metodo	Endpoint	Accesso	Descrizione
POST	/api/auth/login	Anonimo	Verifica userName/password, restituisce token JWT + scadenza + displayName.
POST	/api/auth/logout	Autenticato	Inserisce il jti del token corrente in blacklist; risponde 204 No Content.
POST	/api/auth/register	Admin	Crea un nuovo dipendente con ruolo User di default.

Worklog

Route condivise tra Admin e utente normale: l'Admin accede a tutti i worklog con filtri liberi, l'utente normale solo ai propri (IdEmployee forzato dal claim del token, mai dal body).

Metodo	Endpoint	Accesso	Descrizione
GET	/api/worklog	Autenticato	Elenco worklog, filtrabile (Admin) per employeeId/projectId/dateFrom/dateTo/statusName.
GET	/api/worklog/{id}	Autenticato	Dettaglio di un worklog.
POST	/api/worklog	Autenticato	Crea un worklog.
PUT	/api/worklog/{id}	Autenticato	Aggiorna un worklog.
DELETE	/api/worklog/{id}	Autenticato	Elimina un worklog (cancellazione fisica — vedi §6).

Report e Log

Metodo	Endpoint	Accesso	Descrizione
GET	/api/report-data/project/{projectId}	Admin	Report aggregato per progetto/periodo: totale ore, ripartizione per tipo/stato/giorno/dipendente, worklog di dettaglio.
GET	/api/report-data/employee/{employeeId}	Admin	Report aggregato per un dipendente specifico/periodo.
GET	/api/report-data/employee	Autenticato	Report aggregato del dipendente autenticato/periodo (self-service).
GET	/api/log	Admin	Log applicativo paginato, filtrabile per testo/livello/metodo/intervallo di date.

Anagrafiche (Company, Employee, Project, Status, Type)

Le cinque anagrafiche seguono lo stesso pattern CRUD; Company/Status/Type/Employee(scrittura) sono riservate al ruolo Admin, Project è in lettura per ogni utente autenticato e in scrittura solo per Admin. Employee non espone una POST diretta: la creazione di un dipendente avviene tramite /api/auth/register.

Metodo	Endpoint	Note
GET	/api/company	Elenco aziende
GET	/api/company/{id}	Dettaglio azienda
POST	/api/company	Crea azienda
PUT	/api/company/{id}	Aggiorna azienda
DELETE	/api/company/{id}	Elimina azienda (409 se ha progetti collegati)
GET	/api/employee	Elenco dipendenti (Admin)
GET	/api/employee/{id}	Dettaglio dipendente (Admin)
PUT	/api/employee/{id}	Aggiorna dipendente (Admin)
DELETE	/api/employee/{id}	Elimina dipendente (Admin)
GET	/api/project	Elenco progetti (ogni utente autenticato)
GET	/api/project/{id}	Dettaglio progetto
POST	/api/project	Crea progetto (Admin)
PUT	/api/project/{id}	Aggiorna progetto (Admin)
DELETE	/api/project/{id}	Elimina progetto (409 se ha worklog collegati, Admin)
GET	/api/status	Elenco stati
POST / PUT / DELETE	/api/status/{id}	Gestione stati (Admin)
GET	/api/type	Elenco tipologie
POST / PUT / DELETE	/api/type/{id}	Gestione tipologie (Admin)

4. Visualizzazione Struttura Macro e Micro

4.1 Struttura Macro

Tre progetti backend (API, DataAccess, Domain) con dipendenze rivolte verso l'interno — Domain non dipende da nulla, DataAccess dipende solo da Domain, API dipende da entrambi — più un progetto Frontend (Blazor WebAssembly) che dialoga con l'API esclusivamente via HTTP/REST, senza riferimenti di progetto diretti verso il backend.

4.2 Struttura Micro

4.2.1 Componenti del Layer API

- Controllers — un controller per area (Auth, Worklog, Company, Employee, Project, Status, Type, ReportData, Log), sottili: validano il ModelState, delegano al service applicativo e traducono l'esito in una risposta HTTP.
- Services/ — estensioni di bootstrap: AddXpoInfrastructure (Xpo), AddJwtAuthenticationWithBlacklist (Jwt), AddBlazorClientCors (Cors), AddApiDocumentation (Swagger), AddApplicationServices (DI applicativa).
- helpers/ — UseGlobalProblemDetails (gestione errori), RequestContextEnricher e XpoSerilogSink (logging su DB), ClaimsPrincipalExtensions.

4.2.2 Componenti del Layer DataAccess

- DataContext/ — DbContextService (Session/UnitOfWork XPO) e DataSeeder (dati demo + utente admin).
- Services/ — un service per entità (WorkLogAdminService, WorkLogUserService, CompanyService, EmployeeService, ProjectService, StatusService, TypeService, AuthService, TokenBlacklistService, ReportService, LogService), ciascuno responsabile del proprio CRUD e delle regole di business dirette.
- helpers/ — WorkLogQueryExtensions (filtro Active() per il soft delete) e MappingProfile (AutoMapper, Entity ↔ DTO).

4.2.3 Componenti del Layer Domain

- Entities/ — modelli XPO (Company, Project, Employee, Type, Status, WorkLog, LogApplicativo, BlacklistedToken).
- Dtos/ — un contenitore per entità, con classi annidate Response/Create/Update; WorkLogDto distingue inoltre le viste Admin e User.
- Interfaces/ — un contratto per ciascun service, più IDbContextService e IJwtTokenService.
- Constants/ — RoleNames (Admin/User), centralizzato per evitare stringhe duplicate tra AuthService, DataSeeder e gli attributi [Authorize(Roles = ...)].

4.2.4 Componenti del Frontend (Blazor WebAssembly)

- Pages/Dashboard/ — area utente: griglia worklog, modali di creazione/modifica/eliminazione, pannello e download report.
- Pages/Admin/ — anagrafiche (Companies, Employees, Projects, Statuses, Types) e consultazione Log, ciascuna con tabella + modali dedicati.
- Services/ — client HTTP tipizzati per ogni risorsa (WorkLogApiClient, CompanyApiClient, ProjectApiClient, ecc.), AuthService + CustomAuthenticationStateProvider + TokenStorageService per il ciclo di vita del token, AuthenticatedHttpMessageHandler per l'iniezione automatica dell'header Authorization.
- Services/Reports/ — PdfReportService (composizione del PDF con PDFsharp) e DownloadInterop (modulo JS interop per esporre il blob generato come download).
- Layout/ — MainLayout e componenti di redirect (RedirectToDashboard, RedirectToLogin) coerenti con lo stato di autenticazione.

4.2.5 Flusso di Generazione del Report PDF

Il programma restituisce un report in formato PDF che fotografa lo stato di avanzamento del lavoro svolto in un determinato periodo, in due varianti selezionabili dall'utente: report per tecnico e report per progetto (con il worklog completo di tutti i tecnici coinvolti). La generazione del documento è interamente a carico del client (Blazor WebAssembly): il backend si limita a fornire i dati già aggregati.

1. L'utente, dal pannello report della Dashboard, sceglie tipologia, entità (tecnico o progetto) e periodo, e richiede il download.
2. ReportDataApiClient invoca GET /api/report-data/employee/{id} oppure GET /api/report-data/project/{id}, allegando il token JWT.
3. ReportDataController valida l'intervallo di date e invoca IReportService per l'aggregazione.
4. Il backend restituisce i dati aggregati in JSON (totali, ripartizioni per tipo/stato/giorno/progetto o dipendente, worklog di dettaglio).
5. PdfReportService, eseguito in WASM nel browser, compone il PDF (intestazione, tabella ore, riepilogo) tramite PDFsharp.
6. Il PDF, generato in memoria, viene esposto come download diretto tramite DownloadInterop, senza ulteriori chiamate al backend.

Questa scelta architetturale elimina la dipendenza da una libreria di rendering PDF lato server, mantenendo il backend come puro fornitore di dati stateless.

5. Note Tecniche di Configurazione

5.1 Configurazione JWT (appsettings.json)

```
"Jwt": {  
  "Issuer": "GestionaleRendicontazione.Api",  
  "Audience": "GestionaleRendicontazione.Client",  
  "SecretKey": "<almeno 32 caratteri>",  
  "ExpiryMinutes": 60  
}
```

Il servizio JwtTokenService rifiuta l'avvio se la SecretKey non è configurata o è più corta di 32 caratteri. Il repository contiene una SecretKey segnaposto.

5.2 CORS (appsettings.json)

```
"Cors": {  
  "AllowedOrigins": [ "https://localhost:7210", "http://localhost:5210" ]  
}
```

Le origini consentite corrispondono agli indirizzi su cui gira il client Blazor in sviluppo; vanno aggiornate con il dominio reale in un ambiente diverso da quello locale.

5.3 Connessione al database

```
"ConnectionStrings": {  
  "DefaultConnection": "XpoProvider=SQLite;Data Source=rendicontazione.db;"  
}
```

Lo schema del database viene creato/aggiornato automaticamente all'avvio (AutoCreateOption.DatabaseAndSchema) sulla base delle Entities mappate per riflessione.

5.4 Avvio e seed dati

All'avvio dell'applicazione (Program.cs) viene invocato DataSeeder.SeedAsync, che popola in modo idempotente dati demo (Company, Project, Type, due Status, un WorkLog) e crea un utente admin (userName admin, password Admin123!) se non già presente.